

Lab 2: HTTP Analysis Using Wireshark - Embedded Image Traffic

Forensic Analysis Report

Student name: Afeez Bolaji Usman

Student ID: 202514326WY

Course Code: SBT-DF203

Course Title : Basic Networking Skills for Digital Forensics

Lab Title: Lab2 HTTP Analysis Using Wireshark: Embedded Image Traffic

Instructor Name: Aminu Idris

Date: 9-10/9/2026

13. Executive Summary and Scope

This report documents the capture, analysis, and forensic reconstruction of a plaintext HTTP session involving a webpage with an embedded image, transferred between a Firefox browser client and a local Apache web server on a single Kali Linux virtual machine. The exercise demonstrates that a single page load generates multiple independent HTTP transactions, analyzes how a large object is split across multiple TCP segments and reassembled, and verifies that the image can be fully and accurately recovered from the packet capture alone.

All objectives were met. The browser capture confirmed three separate HTTP GET requests arising from one page visit — the HTML document, the embedded JPEG image, and an automatic browser request for a favicon. The image transfer (68,661 bytes) was found to span three TCP segments totaling 69,150 bytes, with the 489-byte difference accounted for by HTTP response headers, confirming complete reassembly with no data loss. The image was subsequently extracted directly from the packet capture

using tshark's object-export feature, and its SHA-256 hash was found to match the original source file exactly, providing cryptographic proof of byte-for-byte recovery. A separate control capture using curl alone confirmed that command-line tools which lack HTML parsing capability do not automatically retrieve embedded resources, in contrast to browser behavior.

Scope: This exercise was limited to a single local HTTP session over the loopback interface (lo), between one Apache server and one Firefox client, both running on the same authorized virtual machine. No third-party systems, production networks, or public networks were involved at any stage.

Mini Evidence and Chain-of-Custody Worksheet

Field	Entry
Case/lab identifier	SBT-DF203-Lab2-AfeezBolaji
Student name	Afeez Bolaji
Date and time started	9 September 2026, ~17:45 EDT
Evidence file name(s)	evidence/image_traffic.pcapng, evidence/curl_only.pcapng
Source / generation method	Wireshark GUI (browser capture, lo, tcp port 80); tshark foreground capture (curl-only comparison)
Original SHA-256	79acfaa7ebbbcb18d859b2a340d16443c992049776932f2862bdc835569a3dda (image_traffic.pcapng)
Working-copy SHA-256	79acfaa7ebbbcb18d859b2a340d16443c992049776932f2862bdc835569a3dda (matches)
Analysis workstation/VM	Kali Linux (VMware)
Notes on any changes	Two earlier browser-capture attempts discarded: (1) wrong interface (eth0, no traffic); (2) valid interface but browser sent a cached/conditional request yielding 304 Not Modified instead of full object transfer. Final capture used a fresh private window with no prior cache state.

14. Evidence Acquisition and Integrity Hashes

The lab folder structure was created following the standard evidence/working/exported/reports layout.

Apache

was confirmed active and serving the source objects prior to capture.

```
(kali@kali)-[~]
└─$ mkdir -p ~/SBT-DF203-Lab2/{evidence,working,exported,reports,screenshots,scripts}

(kali@kali)-[~]
└─$ cd SBT-DF203-Lab2

(kali@kali)-[~/SBT-DF203-Lab2]
└─$ pwd
/home/kali/SBT-DF203-Lab2

(kali@kali)-[~/SBT-DF203-Lab2]
└─$ find . -maxdepth 1 -type d -print
.
./exported
./evidence
./working
./screenshots
./scripts
./reports

(kali@kali)-[~/SBT-DF203-Lab2]
└─$ tree
.
├── evidence
├── exported
├── reports
├── screenshots
├── scripts
├── working
└── 7 directories, 0 files
```

Figure-1 Lab folder structure created under ~/SBT-DF203-Lab2

```
(kali@kali)-[~/SBT-DF203-Lab2]
└─$ sudo systemctl enable --now apache2

(kali@kali)-[~/SBT-DF203-Lab2]
└─$ systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-09-09 11:51:41 EDT; 1h 20min ago
 Invocation: 999af901f60645cf9b8bb46a0795cfbb
    Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 960 (apache2)
    Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec: 0 B/sec"
     Tasks: 6 (limit: 4403)
  Memory: 18.4M (peak: 23.5M)
     CPU: 2.065s
   CGroup: /system.slice/apache2.service
           └─ 960 /usr/sbin/apache2 -k start -DFOREGROUND
              1001 /usr/sbin/apache2 -k start -DFOREGROUND
              1002 /usr/sbin/apache2 -k start -DFOREGROUND
              1003 /usr/sbin/apache2 -k start -DFOREGROUND
              1004 /usr/sbin/apache2 -k start -DFOREGROUND
              1005 /usr/sbin/apache2 -k start -DFOREGROUND
```

Figure 2 — Apache service confirmed active and running.

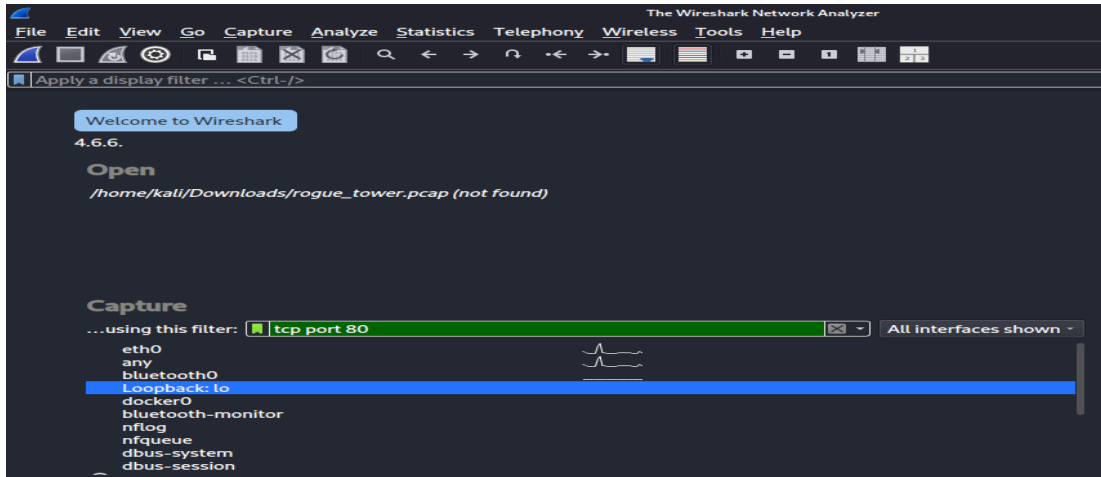


Figure 3 — Wireshark interface selection: Loopback (lo), capture filter tcp port 80.

Source object baseline hashes were recorded prior to capture, establishing the reference values against which recovered objects were later compared (Section 18)

```
(kali@kali)-[~/SBT-DF203-Lab2]
$ sha256sum /var/www/html/image.html /var/www/html/lab_photo.jpg | tee reports/source_object_hashes.txt
7819657651ecd8c9da048b06489b9ba64ca70876c1dbf0c77d91403b9a07e0a5 /var/www/html/image.html
a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeef97afdb /var/www/html/lab_photo.jpg

(kali@kali)-[~/SBT-DF203-Lab2]
$
```

Figure 4 — SHA-256 hashes of source objects (image.html, lab_photo.jpg).

```
kali@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help

(kali@kali)-[~/SBT-DF203-Lab2]
$ cp --preserve=timestamps evidence/image_traffic.pcapng working/image_traffic_working.pcapng

(kali@kali)-[~/SBT-DF203-Lab2]
$ cp -p evidence/image_traffic.pcapng working/image_traffic_working.pcapng

(kali@kali)-[~/SBT-DF203-Lab2]
$ sha256sum evidence/image_traffic.pcapng working/image_traffic_working.pcapng | tee reports/capture_hashes.txt
79acfaa7ebbbcb18d859b2a340d16443c992049776932f2862bdc835569a3dda evidence/image_traffic.pcapng
79acfaa7ebbbcb18d859b2a340d16443c992049776932f2862bdc835569a3dda working/image_traffic_working.pcapng

(kali@kali)-[~/SBT-DF203-Lab2]
$
```

Figure 5 — Working copy created and hash-verified against the original evidence file.

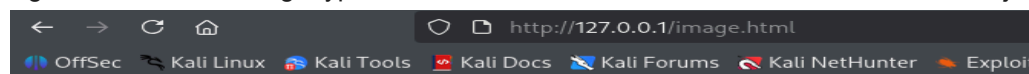
Part A - Verify the Webpage and Eliminate Cache Effects

```
(kali@kali)-[~/SBT-DF203-Lab2]
$ file /var/www/html/lab_photo.jpg
identify /var/www/html/lab_photo.jpg
ls -lh /var/www/html/image.html /var/www/html/lab_photo.jpg
/var/www/html/lab_photo.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, baseline, precision 8, 640x415, components 3
/var/www/html/lab_photo.jpg JPEG 640x415 640x415+0+0 8-bit sRGB 68661B 0.000u 0:00.000
-rw-r--r-- 1 root root 149 Sep  9 13:45 /var/www/html/image.html
-rw-r--r-- 1 root root 68K Sep  9 13:45 /var/www/html/lab_photo.jpg

(kali@kali)-[~/SBT-DF203-Lab2]
$ curl -I http://127.0.0.1/image.html
HTTP/1.1 200 OK
Date: Wed, 09 Sep 2026 17:47:26 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Wed, 09 Sep 2026 17:45:55 GMT
ETag: "95-65b106c59b90c"
Accept-Ranges: bytes
Content-Length: 149
Vary: Accept-Encoding
Content-Type: text/html

(kali@kali)-[~/SBT-DF203-Lab2]
$ curl -I http://127.0.0.1/lab_photo.jpg
HTTP/1.1 200 OK
Date: Wed, 09 Sep 2026 17:47:39 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Wed, 09 Sep 2026 17:45:42 GMT
ETag: "10c35-65b106b8a0b31"
Accept-Ranges: bytes
Content-Length: 68661
Content-Type: image/jpeg
```

Figure 6 — Source image type, dimensions, size, and curl header checks for both objects



SBT-DF203 Image Traffic

Analyst: Afeez Bolaji



Figure 7 — image.html rendered in Firefox, showing the embedded photograph and analyst name

15. HTTP Request/Response Inventory

```

kali@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help

(kali@kali)-[~/SBT-DF203-Lab2]
$ wireshark
** (wireshark:114591) 15:28:29.822080 [GUI WARNING] -- Failed to register with host portal QDBusError("org.freedesktop.portal.Error.Failed", "Could not register app ID: Connection already associated with an application ID")
** (wireshark:114591) 15:33:58.643178 [Capture MESSAGE] -- Capture Start ...
** (wireshark:114591) 15:33:58.692884 [Capture MESSAGE] -- Capture started
** (wireshark:114591) 15:33:58.692918 [Capture MESSAGE] -- File: "/tmp/wireshark_LoopbackRANRV3.pcapng"
** (wireshark:114591) 17:25:52.428213 [GUI WARNING] -- AtSpiAdaptor::applicationInterface does not implement "GetApplicationBusAddress" "/org/a11y/atspi/accessible/root"
^C

(kali@kali)-[~/SBT-DF203-Lab2]
$

```

Figure 8 — tshark console log confirming capture start on the correct interface.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	74	60526 → 80 [SYN] Seq=0 Win=65495
2	0.001627930	127.0.0.1	127.0.0.1	TCP	74	80 → 60526 [SYN, ACK] Seq=0 Ack=1
3	0.001649482	127.0.0.1	127.0.0.1	TCP	66	60526 → 80 [ACK] Seq=1 Ack=1 Win=
4	0.028137907	127.0.0.1	127.0.0.1	HTTP	509	GET /image.html HTTP/1.1
5	0.031529124	127.0.0.1	127.0.0.1	TCP	66	80 → 60526 [ACK] Seq=1 Ack=444 Wi
6	0.117737371	127.0.0.1	127.0.0.1	HTTP	553	HTTP/1.1 200 OK (text/html)
7	0.117811538	127.0.0.1	127.0.0.1	TCP	66	60526 → 80 [ACK] Seq=444 Ack=488
8	0.777475820	127.0.0.1	127.0.0.1	HTTP	509	GET /lab_photo.jpg HTTP/1.1
9	0.777546783	127.0.0.1	127.0.0.1	TCP	66	80 → 60526 [ACK] Seq=488 Ack=887
10	0.781040695	127.0.0.1	127.0.0.1	TCP	32834	80 → 60526 [ACK] Seq=488 Ack=887
11	0.781068647	127.0.0.1	127.0.0.1	TCP	66	60526 → 80 [ACK] Seq=887 Ack=3325
12	0.782482362	127.0.0.1	127.0.0.1	TCP	32834	80 → 60526 [PSH, ACK] Seq=33256 A
13	0.782516460	127.0.0.1	127.0.0.1	TCP	66	60526 → 80 [ACK] Seq=887 Ack=6602
14	0.782576659	127.0.0.1	127.0.0.1	HTTP	3480	HTTP/1.1 200 OK (JPEG JFIF image)
15	0.782818941	127.0.0.1	127.0.0.1	TCP	66	60526 → 80 [ACK] Seq=887 Ack=6943
16	0.834134542	127.0.0.1	127.0.0.1	HTTP	504	GET /favicon.ico HTTP/1.1
17	0.835729668	127.0.0.1	127.0.0.1	TCP	66	60526 → 80 [FIN, ACK] Seq=1325 Ac
18	0.840263382	127.0.0.1	127.0.0.1	HTTP	593	HTTP/1.1 404 Not Found (text/htm
19	0.840302052	127.0.0.1	127.0.0.1	TCP	54	60526 → 80 [RST] Seq=1326 Win=0

Encapsulation type: Ethernet (1)
Arrival Time: Sep 9, 2026 17:46:49.144303620 EDT
UTC Arrival Time: Sep 9, 2026 21:46:49.144303620 U
Epoch Arrival Time: 1788990409.144303620
[Time shift for this packet: 0.000000000 seconds]
[Time since reference or first frame: 0.000000000 s]

Figure 9 — Full 19-packet browser session: handshake, three HTTP requests/responses, and connection reset.

Frame 4 → 6: GET /image.html → 200 OK (text/html)

Frame 8 → 14: GET /lab_photo.jpg → 200 OK (JPEG JFIF image), spanning two TCP segments of 32,834 bytes each (frames 11, 13)

```
(kali@kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -Y 'http.request' -T fields \
-e frame.number -e frame.time -e tcp.stream -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e http.request.method -e http.request.uri -e http.host \
| tee reports/http_object_requests.tsv
4      2026-09-09T17:46:49.172441527-0400      0      127.0.0.1      60526      127.0.0.1      80      GET      /image.html
8      2026-09-09T17:46:49.921779440-0400      0      127.0.0.1      60526      127.0.0.1      80      GET      /lab_photo.jpg
16     2026-09-09T17:46:49.978438162-0400      0      127.0.0.1      60526      127.0.0.1      80      GET      /favicon.ico

(kali@kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -Y 'http.response' -T fields \
-e frame.number -e frame.time -e tcp.stream -e http.response.code -e http.content_type -e http.content_length \
| tee reports/http_object_responses.tsv
6      2026-09-09T17:46:49.262040991-0400      0      200      text/html      151
14     2026-09-09T17:46:49.926880279-0400      0      200      image/jpeg      68661
18     2026-09-09T17:46:49.984567002-0400      0      404      text/html; charset=iso-8859-1      311

(kali@kali)-[~/SBT-DF203-Lab2]
$
```

Figure 10 — tshark field extraction of all HTTP requests and responses. The image's reported Content-Length (68,661 bytes) matches the source file size confirmed in Part A exactly. The 404 response for /favicon.ico is expected: Apache has no favicon configured for this site, and browsers automatically request this file on every page load regardless of whether the page references one — this is documented rather than treated as an error, per the lab's instructions.

16. TCP Segmentation and Reassembly Analysis

Frame	Tcp payload	Range
10	32,768 bytes	488 → 33,256
11	32,768 bytes	33,256 → 66,024
14	3,414 bytes	66,024 → 69,438
Total	69,150 bytes	

Summed TCP payload across the three segments (69,150 bytes) minus the known source image size (68,661 bytes) leaves 489 bytes, corresponding to the HTTP response headers (Date, Server, Last-Modified, ETag, Accept-Ranges, Content-Length, Content-Type) prepended in the first data segment. This confirms the full object was reassembled with no data loss.

```
(kali@kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -Y 'tcp.stream==0 66 tcp.len>0' -T fields -e frame.number -e frame.time -e ip.len -e tcp.hdr.len -e tcp.len -e tcp.seq -e tcp.ack | tee reports/image_stream_segments.tsv.
4      2026-09-09T17:46:49.172441527-0400      495      32      443      1      1
6      2026-09-09T17:46:49.262040991-0400      539      32      487      1      444
8      2026-09-09T17:46:49.921779440-0400      495      32      443      444      488
10     2026-09-09T17:46:49.925344315-0400      32820     32      32768      488      887
12     2026-09-09T17:46:49.926785982-0400      32820     32      32768      33256      887
14     2026-09-09T17:46:49.926880279-0400      3466      32      3414      66024      887
16     2026-09-09T17:46:49.978438162-0400      490      32      438      887      69438
18     2026-09-09T17:46:49.984567002-0400      579      32      527      69438      1326
```

Figure 11 — tshark extraction of all data-carrying segments in the stream.

```
(kali@kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -q -z conv,tcp | tee reports/tcp_conversations.txt

TCP Conversations
Filter:<No Filter>

Relative | Duration | | | | Total |
Start | | | Frames Bytes | | Frames Bytes | | Frames Bytes |
127.0.0.1:60526 | | | 8 70 kB | | 11 2,046 bytes | | 19 72 kB
0.000000000 | 0.8403 | | | | | | |
```

Figure 12 — TCP conversation summary: 19 packets, 70 kB server→client, 2,046 bytes client→server

The reference lab material illustrates this transfer split across two segments; this capture instead produced three (two full 32,768-byte segments plus a smaller remainder). This variation is expected and does not indicate an error — TCP segmentation is determined dynamically by factors such as operating system send-buffer size, network interface offloading behavior, and the effective maximum segment size negotiated for the connection, none of which are guaranteed to reproduce identically across different systems or capture environments.

17. Object Extraction Method

HTTP objects were exported directly from the working capture using tshark's built-in object-export feature, which reassembles and writes out every complete HTTP object found in the capture. mkdir -p exported/http_objects tshark -r working/image_traffic_working.pcapng --export-objects http,exported/http_objects

```
$ mkdir -p exported/http_objects

(kali@kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng --export-objects http,exported/http_objects
1 0.000000000 127.0.0.1 → 127.0.0.1 TCP 74 60526 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSv
al=3250844194 TSecr=0 WS=512
2 0.001627930 127.0.0.1 → 127.0.0.1 TCP 74 80 → 60526 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SA
CK_PERM TSval=473978469 TSecr=3250844194 WS=512
3 0.001649482 127.0.0.1 → 127.0.0.1 TCP 66 60526 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=3250844196
TSecr=473978469
4 0.028137907 127.0.0.1 → 127.0.0.1 HTTP 509 GET /image.html HTTP/1.1
5 0.031529124 127.0.0.1 → 127.0.0.1 TCP 66 80 → 60526 [ACK] Seq=1 Ack=444 Win=65536 Len=0 TSval=473978498
TSecr=3250844222
6 0.117737371 127.0.0.1 → 127.0.0.1 HTTP 553 HTTP/1.1 200 OK (text/html)
7 0.117811538 127.0.0.1 → 127.0.0.1 TCP 66 60526 → 80 [ACK] Seq=444 Ack=488 Win=65536 Len=0 TSval=3250844
312 TSecr=473978585
8 0.777475820 127.0.0.1 → 127.0.0.1 HTTP 509 GET /lab_photo.jpg HTTP/1.1
9 0.777546783 127.0.0.1 → 127.0.0.1 TCP 66 80 → 60526 [ACK] Seq=488 Ack=887 Win=65536 Len=0 TSval=4739792
45 TSecr=3250844972
10 0.781040695 127.0.0.1 → 127.0.0.1 TCP 32834 HTTP/1.1 200 OK
11 0.781068647 127.0.0.1 → 127.0.0.1 TCP 66 60526 → 80 [ACK] Seq=887 Ack=33256 Win=94720 Len=0 TSval=32508
44975 TSecr=473979248
12 0.782482362 127.0.0.1 → 127.0.0.1 TCP 32834 80 → 60526 [PSH, ACK] Seq=33256 Ack=887 Win=65536 Len=32768
TSval=473979250 TSecr=3250844972
13 0.782516460 127.0.0.1 → 127.0.0.1 TCP 66 60526 → 80 [ACK] Seq=887 Ack=66024 Win=61952 Len=0 TSval=32508
44977 TSecr=473979250
14 0.782576659 127.0.0.1 → 127.0.0.1 HTTP 3480 HTTP/1.1 200 OK (JPEG JFIF image)
15 0.782818941 127.0.0.1 → 127.0.0.1 TCP 66 60526 → 80 [ACK] Seq=887 Ack=69438 Win=94720 Len=0 TSval=32508
44977 TSecr=473979250
16 0.834134542 127.0.0.1 → 127.0.0.1 HTTP 504 GET /favicon.ico HTTP/1.1
17 0.835729668 127.0.0.1 → 127.0.0.1 TCP 66 60526 → 80 [FIN, ACK] Seq=1325 Ack=69438 Win=94720 Len=0 TSval
=3250845030 TSecr=473979250
18 0.840263382 127.0.0.1 → 127.0.0.1 HTTP 593 HTTP/1.1 404 Not Found (text/html)
19 0.840302053 127.0.0.1 → 127.0.0.1 TCP 54 60526 → 80 [RST] Seq=1326 Win=0 Len=0
```

Figure 13 — tshark export-objects command processing the full capture

```
(kali@kali)-[~/SBT-DF203-Lab2]
└─$ find exported/http_objects -maxdepth 1 -type f -ls
file exported/http_objects/*
 3565558    68 -rw-r--r--   1 kali   kali      68661 Sep  9 18:53 exported/http_objects/lab_photo.jpg
 3565559     4 -rw-r--r--   1 kali   kali        311 Sep  9 18:53 exported/http_objects/favicon.ico
 3565557     4 -rw-r--r--   1 kali   kali        149 Sep  9 18:53 exported/http_objects/image.html
exported/http_objects/favicon.ico:  HTML document, ASCII text
exported/http_objects/image.html:  HTML document, ASCII text
exported/http_objects/lab_photo.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length
16, baseline, precision 8, 640x415, components 3

(kali@kali)-[~/SBT-DF203-Lab2]
└─$ sha256sum /var/www/html/lab_photo.jpg exported/http_objects/* | tee reports/extracted_object_hashes.txt
a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeef97afdb /var/www/html/lab_photo.jpg
df8c73cf6bd614e7f204500ace411fe52f0ea6651be6b0241185e740a8f35d7d exported/http_objects/favicon.ico
7819657651ecd8c9da048b06489b9ba64ca70876c1dbf0c77d91403b9a07e0a5 exported/http_objects/image.html
a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeef97afdb exported/http_objects/lab_photo.jpg
```

Figure 14 — Extracted files confirmed by type: lab_photo.jpg (JPEG), image.html and favicon.ico (both HTML text).

Three files were recovered: lab_photo.jpg (68,661 bytes, JPEG image data), image.html (149 bytes), and a file named favicon.ico (311 bytes). The file command confirms the third file is not actually image/icon data — it is the HTML body of the 404 Not Found response Apache returned for the unconfigured favicon request, saved by tshark under the name of the object that was requested rather than its actual content type.

18. Hash-Based Integrity Conclusion

File	Original SHA-256	Extracted SHA-256	Match
lab_photo.jpg	a2143e21d1f93...97afd b	a2143e21d1f93...97afd b	YES
image.html	7819657651ec...9a07e 0a5	7819657651ec...9a07e 0a5	YES

Conclusion: both objects were recovered byte-for-byte identical to their source files, proving that plaintext HTTP traffic can be fully reconstructed from a packet capture with cryptographic proof of integrity. A different filename (as with the favicon/404 file above) does not itself indicate an error — the determining factor is whether size, type, and hash all agree with the expected source object.

19. curl versus Browser Explanation

A separate, isolated capture was taken while requesting image.html using curl alone (no browser involved), to compare against the browser-generated traffic analyzed above.

```

(kali@kali)-[~/SBT-DF203-Lab2]
$ tshark -r evidence/curl_only.pcapng

(kali@kali)-[~/SBT-DF203-Lab2]
$ curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
* Trying 127.0.0.1:80 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 80)
) from 127.0.0.1 port 49778
% Total    % Received % Xferd  Average Speed   Time
Time      Time     Current           Dload  Upload   Total
Spent     Left     Speed
0 0 0    0 0 0      0 0 0      0 0 0      0 0 0
0* using HTTP/1.x
> GET /image.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Wed, 09 Sep 2026 23:20:44 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Wed, 09 Sep 2026 17:45:55 GMT
< ETag: "95-65b106c59b90c"
< Accept-Ranges: bytes
< Content-Length: 149
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [149 bytes data]
100 149 100    149 0    0    2713    0
100 149 100    149 0    0    2688    0
100 149 100    149 0    0    2678    0
0
* Connection #0 to host 127.0.0.1:80 left intact

(kali@kali)-[~/SBT-DF203-Lab2]
$ sudo tshark -i lo -f 'tcp port 80' -w /tmp/curl_only.pcapng
[sudo] password for kali:
Running as user "root" and group "root". This could be dangerous.
Capturing on 'Loopback: lo'
10 ^C

(kali@kali)-[~/SBT-DF203-Lab2]
$ sudo mv /tmp/curl_only.pcapng evidence/curl_only.pcapng
sudo chown kali:kali evidence/curl_only.pcapng
tshark -r evidence/curl_only.pcapng -Y 'http.request' -T fields -e http.request.uri | tee reports/curl_requested_objects.txt /image.html

(kali@kali)-[~/SBT-DF203-Lab2]
$

(kali@kali)-[~/SBT-DF203-Lab2]
$

(kali@kali)-[~/SBT-DF203-Lab2]
$

(kali@kali)-[~/SBT-DF203-Lab2]
$

```

Figure 15 — curl-only capture and request extraction, showing a single GET request with no follow-up fetches.

Client	Request Generate	Object fetched
curl (command-line)	1	/image.html only
3	Firefox (browser)	/image.html, /lab_photo.jpg, /favicon.ico

This confirms the fundamental behavioral difference between the two tools: curl retrieves only the exact URL it is given and has no HTML parser — it treats the response body as opaque data and does not inspect it for further references. A web browser, by contrast, parses the HTML it receives, identifies embedded resource tags such as ``, and automatically issues additional requests to fetch each referenced object. This is why loading a single webpage in a browser can generate many HTTP transactions, while the equivalent curl request generates exactly one.

20. Limitations and Recommendations

This capture was performed entirely on the loopback interface, so it does not reflect real network conditions (latency, packet loss, real Ethernet framing) and shows no genuine MAC addresses. Two capture attempts were discarded before a valid result was obtained: one due to selecting the wrong network interface, and one due to browser caching producing a 304 Not Modified response instead of a full object transfer. For future exercises of this type, it is recommended to always begin from a completely fresh private-browsing session with no prior visits to the target URL, to avoid conditional-GET artifacts suppressing the object transfer needed for analysis. Additionally, automated object-extraction tools may

name recovered files after the requested URI rather than their actual content — as seen with the favicon.ico file in this capture — and this should always be verified with the file command rather than assumed from the filename alone.

Conclusion

This exercise successfully demonstrated that a single webpage load generates multiple independent HTTP transactions, that large objects are transparently segmented and reassembled at the TCP layer without loss, and that transferred objects can be fully recovered from a packet capture with cryptographic proof of integrity via SHA-256 hash comparison. The curl-versus-browser comparison further clarified that HTTP object discovery is a browser-side behavior driven by HTML parsing, not an inherent property of the HTTP protocol itself.

Required Forensic Findings

Finding	Value / Observation
Number of HTTP requests	3 (image.html, lab_photo.jpg, favicon.ico)
HTML request URI and timestamp	/image.html, 17:46:49.172 EDT
Image request URI and timestamp	/lab_photo.jpg, 17:46:49.922 EDT
HTTP response codes	200, 200, 404
Image content type	image/jpeg
Reported Content-Length	68,661 bytes
Number of reassembled TCP segments	3
Sum of TCP payload lengths	69,150 bytes
Original image SHA-256	a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeef97afd b
Extracted image SHA-256	a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeef97afd b
Hash comparison conclusion	Identical — full byte-for-byte recovery confirmed
curl/browser difference	curl: 1 request; browser: 3 requests (parses HTML, follows embedded references)

Appendix — Screenshot and Command Reference

Item	Status
Webpage showing the embedded image	Confirmed — Figure 7
Source image type, dimensions, size and hash	Confirmed — Figures 4, 6
Capture of the browser request	Confirmed — Figures 3, 8
HTTP request list showing HTML and image GETs	Confirmed — Figures 9, 10
Image response headers	Confirmed — Figure 10 / http_object_responses.tsv
Reassembled TCP segment list	Confirmed — Figures 11, 12
Export Objects window	tshark export used(Figures 13, 14) in place of GUI dialog
Recovered image opened successfully	Confirmed — file type match, Figure 14
Original and recovered SHA-256 comparison	Confirmed — Section 18
curl-only capture showing requested objects	Confirmed — Figure 15

Full command outputs are available in: reports/source_object_hashes.txt, reports/capture_hashes.txt, reports/http_object_requests.tsv, reports/http_object_responses.tsv, reports/image_stream_segments.tsv, reports/tcp_conversations.txt, reports/extracted_object_hashes.txt, reports/curl_requested_objects.txt.

Student Declaration

I confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions. Date: 10/9/2026